

Laboratory of Mobile Operating Systems

C++ in Android

Introduction

This exercise will teach you how to use a native C++ library inside your own Android application and communicate with it through the Java Native Interface (JNI). Luckily, modern versions of Android Studio should make this task relatively painless, as long as CMake and the Native Development Kit (NDK) are properly installed.

Task 0.

Launch Android Studio and create a new project based on the Native C++ template.

While the project is being built, have a look at the contents of the following files:

- **MainActivity.kt**, which contains logic for loading a sample native library, as well as the declarations of its functions;
- **native-lib.cpp**, which contains all native code, including function definitions;
- **CMakeLists.txt**, which defines the build process of all native libraries used in your project;
- **build.gradle** for the *app* module, which contains additional settings used by CMake.

Task 1. (1 point)

Program your app to make it capable of generating an array of at least 100 random numbers upon user request and displaying its contents on the screen. You can use Kotlin or Java for this, there's no need to go native yet.

Task 2. (2 points)

Create a C++ function that will accept the randomized array as input and sort it. Update your app with an option for calling your sorting function, as well as displaying both the input array and its sorted version.

Variant I

Sort the array in ascending order.

Variant II

Sort the array in descending order.

In your Kotlin code, a native function declaration can be as simple as this:

```
external fun sortArray(array: IntArray)
```

In your C++ library, the native function definition must be properly named after the Kotlin class' package and all of its parameters must be translated to a corresponding JNI type. What is more, non-primitive types require special handling:

```
extern "C"  
JNIEXPORT void JNICALL  
Java_ksg_mso_cpp_MainActivity_sortArray(JNIEnv *env, jobject  
thiz, jintArray array) {  
    int length = env->GetArrayLength(array);  
    jint *arrayElements = env->GetIntArrayElements(array, NULL);  
    // Sort arrayElements here.  
    env->ReleaseIntArrayElements(array, arrayElements, 0);  
}
```

Task 3. (2 points)

Create a Kotlin/Java version of your C++ sorting function and compare the execution time of both methods, including the amount of time spent on JNI communication.